Ricorsione tail, accumulatori e costo

L07 · Programmazione funzionale · OCaml

Mauro Leoncini

Obiettivi

- Analisi **in-depth** del processo di fold
- Quanto costa una ricorsione, in **spazio** e in **tempo**
- La c.d. **posizione di coda**, e l'accumulatore
- `fold_left` e `fold_right`: quando la scelta è indifferente e quando no
- Perché su un albero le regole cambiano

Ricordiamo la "ripiegatura" (fold) sulle liste

La possiamo scrivere così:

```
let rec fold combine neutral = function
   | [] -> neutral
   | x :: r -> combine x (fold combine neutral r)
```

... ma anche così

```
let rec fold combine l neutral =
   match l with
   | [] -> neutral
   | x :: r -> combine x (fold combine r neutral)
```

Da L06: la ripiegatura degli alberi

L'esercizio 2. Quanti argomenti ha `fold_tree`, e chi lo decide?

```
type 'a tree = Leaf | Node of 'a tree * 'a * 'a tree
```

Due costruttori, quindi **due** argomenti — ma `Node` ne ha tre, di cui due ricorsivi: la funzione combinante prende **due** risultati parziali, non uno.

```
let rec fold_tree on_leaf on_node = function
  | Leaf -> on_leaf
  | Node (left, value, right) ->
      on_node
        (fold_tree on_leaf on_node left)
        value
        (fold_tree on_leaf on_node right)
```

Sulla ricorsione strutturale

Sulle liste, e anche sugli alberi, quanti sono i costruttori della struttura, tanti sono i parametri di fold (oltre naturalmente alla stessa struttura **da ripiegare**).

Si tratta di una **coincidenza**?

Vediamo un altro esempio, il fold del tipo espressione di L05

```
type expr =
  | Num of int
  | Add of expr * expr
  | Mul of expr * expr
  | Neg of expr
```

Fold di un'espressione

```
let rec fold_expr on_num on_add on_mul on_neg e =
  let go = fold_expr on_num on_add on_mul on_neg in
  match e with
  | Num n -> on_num n                    (* one branch *)
  | Add (a, b) -> on_add (go a) (go b)   (* per *)
  | Mul (a, b) -> on_mul (go a) (go b)   (* constructor *)
  | Neg a -> on_neg (go a)
```

QUATTRO argomenti, ancora una volta uno per costruttore.

Il conteggio non sembra una coincidenza: un fold è un match dei costruttori trasformati in parametri, e il match ha esattamente un ramo per costruttore – questa è la completezza che abbiamo discusso in L04.

Generalità del fold

L'enunciato preciso:

> Ogni funzione definita per ricorsione strutturale si può scrivere come **istanza di fold**.

fold **non è** una funzione ricorsiva fra tante, è la più generale — e uno slot per costruttore è anche necessario, perché con meno non distingueremmo i rami.

Esistono controesempi?

Sì, se il ramo ricorsivo usa il pezzo originale e non il fold calcolato su di esso

```
let rec insert x = function
  | [] -> [x]
  | y :: r -> if x <= y then x :: y :: r else y :: insert x r
```

Come possiamo ricondurre insert al pattern genereale di fold su lista? Ovvero:

```
let rec fold combine singleton = function
  | [] -> singleton
  | y :: r -> combine y (fold combine singleton r))
```

La parte non ricorsiva è facile: `lista vuota` → `singleton = [x]`

Il problema è la parte ricorsiva perché la costruzione del risultato a volte (cioè se $x > y$) richiede effettivamente di richiamare **insert sulla coda r** mentre altre volte (se $x \leq y$) richiede **semplicemente la coda r**.

`insert` è davvero un controesempio?

In realtà possiamo ricondurre anche la `insert` al pattern computazionale descritto da fold usando coppie:

```
let insert_via_fold x l =
  snd (fold (fun y (orig, res) ->
               (y :: orig, if x <= y then x :: y :: orig
                                     else y :: res))
            ([], [x])
            l)
```

dove, in questo caso, combine è una **chiusura** (**x** non è locale a combine)

Esercizio

```
let rec weird_fold combine zero k = function
  | [] -> zero
  | x::r -> (combine x (weird_fold combine zero k r)) + k
```

Tre parametri per i due costruttori di lista

1. Cosa restituisce `weird_fold ( + ) 0 3 [3; 6; 19; 25]` ?
2. Può essere riscritta come fold canonico (in questo caso con due soli parametri)? Se sì come?

Dove invece fold non si applica affatto

Quando la ricorsione **non è strutturale**, cioè non va sui pezzi immediati del costruttore.

Un caso notevole è il *quicksort*

Nel *quicksort* la ricorsione si fa su una porzione di elementi determinati dalla grandezza del pivot relativamente agli altri elementi

Tale porzione è una sottolista ma non un campo di ::

In questo caso l'affermazione «un argomento per costruttore» non ha nessun significato, perché la forma della ricorsione non è più la forma del tipo

La domanda rimasta aperta

`fold` costruisce la risposta **risalendo**.

Se `l = [1; 2; 3]` il calcolo viene effettuato nel modo seguente

`fold ( + ) l 0 = ( + ) 1 (( + ) 2 (( + ) 3 0))`

L'espressione **cresce** per metà percorso e poi si richiude. Al punto più ampio contiene **n addizioni ancora da eseguire**.

Su una lista di un milione di elementi, dove stanno tutte le operazioni in attesa?

Posizione di coda

`sum`, ricorsivamente

```
x + sum r            (* NON in coda: dopo la chiamata c'è il + *)
```

`sum_acc`, ricorsivamente

```
sum_acc (acc + x) r  (* in coda: dopo la chiamata non c'è nulla *)
```

Si dice che una chiamata è **in posizione di coda** quando il suo risultato **è** il risultato della funzione che la contiene.

Allora il chiamante non ha più niente da ricordare, e il suo posto può essere riusato.

L'accumulatore

```
let rec sum_acc acc = function
  | [] -> acc
  | x :: r -> sum_acc (acc + x) r
```

```
sum_acc 0 [1;2;3] → sum_acc 1 [2;3] → sum_acc 3 [3] → sum_acc 6 [] → 6
```

L'espressione **non cresce mai**: ogni passo la sostituisce invece di incapsularla.

Adesso misuriamo. E c'è una sorpresa

```
ocamlopt esplode.ml -o blowup && ./blowup 20000000 134217728
stack limit: 134217728 words (was 134217728)
list length: 20000000
non tail -> 199999990000000
tail     -> 199999990000000
```

Venti milioni di elementi e **nessun errore**, nemmeno nella versione **non** tail.

La ragione: OCaml 5 non usa lo stack di sistema per la ricorsione OCaml, usa uno stack proprio **allocato sull'heap**, che CRESCE quando serve.

Ma non cresce senza fine: un tetto c'è, ed è il runtime a dichiararlo.

```
(Gc.get ()).stack_limit -> 134217728 parole di 8 byte (1GB)
```

Il tetto predefinito di 1GB a dimensioni realistiche non lo si tocca

Se un limite c'è, l'eccezione arriva

```
ocamlc esplode.ml -o blowup_b
OCAMLRUNPARAM=l=1000000 ./blowup_b 100000   →  4999950000
OCAMLRUNPARAM=l=1000000 ./blowup_b 500000   →  Stack_overflow
```

Con lo stesso limite e lo stesso ingresso, la versione con accumulatore risponde `124999750000`.

A parità di limite di risorse, la versione nativa (cioè il file eseguibile ottenuto con ocamlopt) genera stack overflow per valori più elevati di n

Il calcolo della soglia esatta (per entrambe le versioni), da trovare per bisezione, è l'esercizio 1.

Il costo di memoria

`memoria.ml`, n = 10 milioni, misurato dal processo stesso:

```
the list alone          ~237 700 kB
after sum_acc (tail)    ~237 800 kB      (+84 kB)
after sum (non tail)    ~500 800 kB      (+263 MB)
```

263 megabyte per sommare dieci milioni di interi che erano già in memoria: le n addizioni in attesa, a circa 26 byte l'una.

La versione con accumulatore non chiede nulla in più.

Le due "ripiegature" sulle liste (left e right)

Le liste sono strutture lineari: possiamo ripiegare (fold) da destra o sinistra:

```
let rec fold_right combine l final =
  match l with
  | [] -> final
  | x :: r -> combine x (fold_right combine r final)
```

```
let rec fold_left combine acc l =
  match l with
  | [] -> acc
  | x :: r -> fold_left combine (combine acc x) r
```

Due pieghe, e gli argomenti scambiati

```
List.fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
List.fold_left  : ('b -> 'a -> 'b) -> 'b -> 'a list -> 'b
```

```
fold_right f [a;b;c] z  =  f a (fold_right f [b;c] z)
                        =  f a (f b (fold_right f [c] z))
                        =  f a (f b (f c (fold_right f [] z)))
                        =  f a (f b (f c z))
fold_left  f z [a;b;c]  =  fold_left f (f z a) [b,c]
                        =  fold_left f (f (f z a) b) [c]
                        =  fold_left f (f (f (f z a) b) c) []
                        =  f (f (f z a) b) c)
```

La prima è la `fold` di L06, e **non** è in coda. La seconda sì.

Quando coincidono, e quando no

```
fold_right ( + ) [1;2;3] 0 = 6        fold_left ( + ) 0 [1;2;3] = 6
fold_right ( - ) [1;2;3] 0 = 2        fold_left ( - ) 0 [1;2;3] = -6
```

> Se l'operazione è **associativa** e il valore iniziale è il suo **elemento neutro**, le due pieghe danno lo stesso risultato.

In questo caso conviene usare `fold_left`, che non consuma stack.

«Allora uso sempre fold_left»

```
fold_right (fun x acc -> f x :: acc) l []    (* la lista, in ordine *)
fold_left  (fun acc x -> f x :: acc) [] l    (* rovesciata *)
```

Per l'ordine giusto con `fold_left` bisogna rovesciare alla fine: un'altra passata, e il codice dice meno.

`fold_right` segue la struttura del **tipo**. `fold_left` segue la struttura della **macchina**. La scelta è un compromesso, non una regola.

Il costo in tempo: la `rev` di L04

```
let rec naive_rev = function
  | [] -> []
  | x :: r -> naive_rev r @ [x]
```

```
n = 10000    0.354 s          n = 40000   15.629 s
n = 20000    2.270 s          n = 80000  100.144 s
```

Raddoppiando `n`, il tempo si moltiplica per circa **6,4**.

La causa sta in una riga

```
naive_rev r @ [x]
```

`@` **ricopia tutto il suo primo argomento**. A ogni passo si ricopia l'intera lista già rovesciata: 1 + 2 + … + n.

Per contro, `rev_acc` è molto efficiente

```
let rec rev_acc acc = function
  | [] -> acc
  | x :: r -> rev_acc (x :: acc) r
```

```
x :: acc      (* una cella *)
acc @ [x]     (* tante celle quanto è lunga acc *)
```

> Se dentro una ricorsione sulle liste compare un `@`, fermatevi e contate.

Sugli alberi cambia tutto

Due alberi da **due milioni di nodi** ciascuno, stesso limite di stack:

```
balanced, height 21           size = 2097151
degenerate, 2 000 000 nodes   Stack_overflow
```

Stessa dimensione, esito opposto. Non conta la dimensione: conta l'**altezza**.

Perché è una buona notizia

- lista: lo spazio della ricorsione è lineare nella lunghezza, **sempre**
- albero: dipende dalla **forma**, e su un albero bilanciato l'altezza cresce come il logaritmo

Un albero bilanciato di un miliardo di nodi ha altezza 30.

E `size` non si rende tail aggiungendo un accumulatore: la ricorsione è **doppia**, e servirebbe gestire a mano una pila dei sottoalberi — cioè riscrivere quello che lo stack faceva già.

E il `fold_left` degli alberi?

Su una lista c'erano due pieghe. Su un albero l'analogia si **sdoppia**, lungo due assi che sulle liste coincidevano:

- la direzione in cui viaggia l'**informazione**: dal basso verso l'alto (i risultati dei figli arrivano al padre) oppure dall'alto verso il basso (un parametro scende dalla radice)
- se la ricorsione è **in coda** o no

`fold_tree` è l'analogo esatto di `fold_right`: segue la forma del tipo, e non è in coda.

```
let rec fold_tree on_leaf on_node = function
  | Leaf -> on_leaf
  | Node (left, value, right) ->
      on_node
        (fold_tree on_leaf on_node left)
        value
        (fold_tree on_leaf on_node right)
```

Dall'alto verso il basso

«Per ogni elemento, la sua profondità.» La profondità di un nodo non è un fatto sui suoi sottoalberi: è un fatto sul cammino dalla **radice**.

```
let depths t =
  let rec go d = function
    | Leaf -> []
    | Node (l, x, r) -> go (d+1) l @ [(x, d)] @ go (d+1) r
  in go 0 t
```

Con `fold_tree` sembra impossibile: `on_node` riceve i risultati dei figli, non la propria profondità.

Ripiegare su una funzione

```
let depths_via_fold t =
  fold_tree
    (fun _ -> [])
    (fun fl x fr -> fun d -> fl (d+1) @ [(x, d)] @ fr (d+1))
    t 0
```

Ogni sottoalbero T_x di radice x si ripiega non in un valore, ma in una **funzione** $f_x(d)$.

Se chiamata con argomento la profondità di x , tale funzione produce esattamente il fold del sottoalbero T_x

Il risultato del fold ha tipo `int -> (elemento * int) list`.

In coda, sugli alberi

```
let fold_elements f z t =
  let rec go acc = function
    | [] -> acc
    | Leaf :: rest -> go acc rest
    | Node (l, x, r) :: rest -> go (f acc x) (l :: r :: rest)
  in go z [t]
```

In coda davvero. Il prezzo è `rest`, la lista dei sottoalberi da visitare: **esattamente quello che faceva lo stack**, gestito a mano.

E qualcosa si è perso: `f : 'a -> 'b -> 'a` non vede più i **risultati dei figli**. `sum` e `count` si scrivono, `height` no (almeno non facilmente).

Il costo si è spostato, non è sparito

Con `OCAMLRUNPARAM=l=1000000`:

```
balanced 2^20-1  bottom-up      1048575
balanced 2^20-1  accumulator    1048575
degenerate 10^6  bottom-up      Stack_overflow
degenerate 10^6  accumulator    1000000

lista di lavoro, massimo raggiunto:
  balanced   (altezza 20)  : 21
  degenerate (altezza 10^6): 1000001
```

Sul degenere la versione in coda sopravvive — con **un milione di celle** nella lista di lavoro.

Che cosa portate a casa oggi

- La riduzione di L03 non serve solo a capire: serve a **prevedere un costo**
- Posizione di coda = al chiamante non resta niente da ricordare
- Su OCaml 5 la ricorsione non tail **non esplode**: consuma spazio lineare, in silenzio
- `fold_left` è gratis solo se l'operazione è associativa col neutro giusto
- Un `@` dentro una ricorsione è quasi sempre un errore di costo
- Sugli alberi il costo lo decide l'**altezza**, cioè la forma del dato

Al calcolatore

```
lezioni/L07-ricorsione-tail/codice/
  coda.ml             le due riduzioni, affiancate
  esplode.ml          il comando con OCAMLRUNPARAM, e perché serve
  memoria.ml          237 MB contro 500 MB, misurati
  sinistra_destra.ml  ( + ) e ( - ), e perché non è lo stesso
  costo.ml            i cento secondi di naive_rev
  alberi.ml           balanced contro degenerate
  fold_albero.ml      il fold degli alberi, i due assi, il costo spostato
  esercizi.ml         cinque esercizi, il terzo e il quinto senza codice
```

Riferimenti

- Esercizio da fare prima di L08: il punto 2, con l'ordine del risultato **previsto per iscritto** prima di eseguire
- Domanda aperta, per L08: `rev_acc [] l` e `naive_rev l` calcolano davvero la stessa cosa per **ogni** `l`? Provarlo su qualche esempio non è una risposta.